

System Design Document

Intelligent Inventory Dashboard

Architectural Plan & Technical Design Reference

Prepared for: Dealership Vehicle Inventory Management Coding Challenge

Date: August 8, 2026

1. Overview

The Intelligent Inventory Dashboard is a React single-page application that helps dealership managers monitor vehicle inventory, identify vehicles that have aged past a 90-day threshold, and assign follow-up actions to those vehicles. This document describes the system's architecture, the responsibilities of each component, how data moves through the application, the technology choices behind the implementation, and the strategy for observability so the system remains debuggable and measurable as it grows.

2. Architecture Diagram

The diagram below shows the client-side architecture: how UI components, a dedicated Logic Layer, client state (Redux Toolkit), server state (React Query), the HTTP layer (Axios), and the mocked/real backend (MSW / future API) interact, with an observability layer capturing events across the stack. The Logic Layer sits directly beneath the UI so that business rules — such as aging-stock evaluation, filter/sort logic, and action-status validation — are handled outside of components, in dedicated, independently testable hooks.

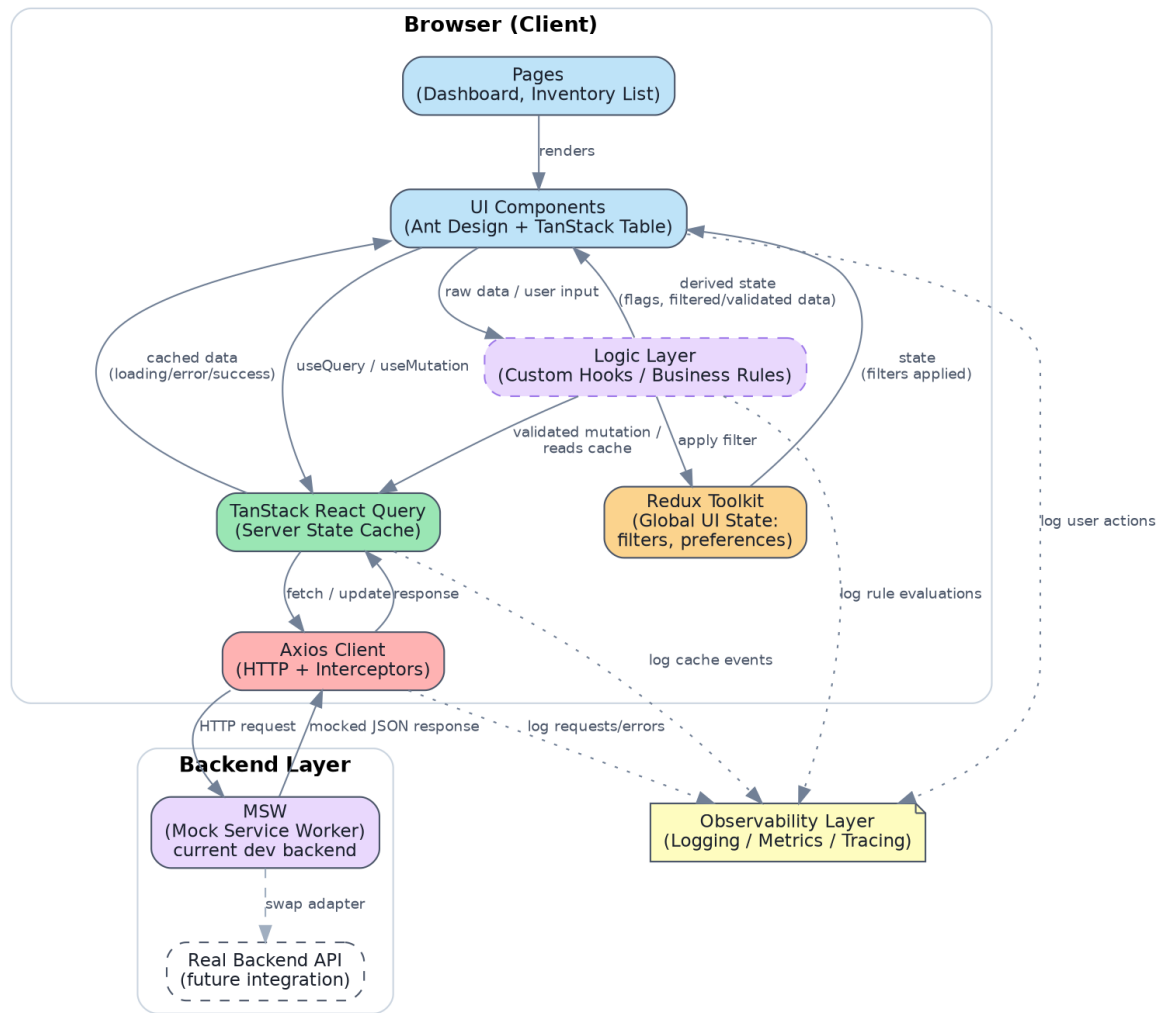


Figure 1. Client-side architecture and data-flow overview, including the Logic Layer beneath the UI.

3. Component Roles

Each layer of the application has a single, well-defined responsibility, which keeps presentation, state management, and data-fetching concerns independent and individually testable.

Component	Role
Pages	Top-level route/screen components (e.g., Inventory Dashboard) that compose feature components and own page-level data fetching triggers.
UI Components (Ant Design + TanStack Table)	Render the inventory table, filter controls, aging-stock highlighting, and the action-assignment modal; presentation logic only, no direct data fetching or business rules.
Logic Layer (Custom Hooks / Business Rules)	Sits directly beneath the UI and holds application business logic —filtering, and validation — in custom hooks, separate from presentation and from server/client state management.
Redux Toolkit Store	Holds global, UI-level client state such as active filters and user preferences that must persist across components without prop drilling.

TanStack React Query	Owns all server-state: fetches vehicle inventory, caches responses, manages loading/error/success states, and drives mutations for action-status updates.
Axios Client	Centralized HTTP layer with interceptors for consistent request/response handling, error normalization, and headers (including future trace propagation).
MSW (Mock Service Worker)	Intercepts network requests during development and returns realistic mock inventory data, standing in for a real backend with minimal code impact when swapped out.
Backend API (future)	Production REST/GraphQL service that will replace MSW handlers behind the same Axios contract, requiring no changes to components or React Query hooks.
Observability Layer (future)	Cross-cutting layer capturing logs, metrics, and traces from the Axios interceptors, React Query cache events, and key user interactions.

4. Data Flow

Reading inventory data:

- The Dashboard page mounts and a React Query hook (e.g., `useQuery`) requests the vehicle list.
- Axios issues the HTTP request through a shared client configured with interceptors.
- In development, MSW intercepts the request at the network level and returns mock inventory data; in production the same request would reach the real backend API without any component changes.
- React Query caches the raw response and exposes loading, error, and success states to the UI.
- The UI passes the raw data through the Logic Layer, which evaluates the 90-day aging-stock rule and returns aging flags back to the UI.
- UI components render the flagged data through TanStack Table, visually highlighting aging stock.

Filtering:

- User interaction with filter controls (make, model, status) is passed to the Logic Layer rather than being applied directly by the component.
- The Logic Layer applies the filter rules, then dispatches the resulting filter state to the Redux Toolkit store.
- Because filtering is client-side, the table re-renders against the already-cached React Query data without a network round trip.

Updating an action:

- A manager selects an action (e.g., 'Price Reduction Planned') in the Ant Design modal; the selection is first passed to the Logic Layer for validation.
- Once validated, the Logic Layer triggers a React Query mutation (`useMutation`) — invalid selections never reach the network layer.
- Axios sends the update request; MSW persists it in its mock data store (swappable for a real write endpoint later).
- On success, React Query invalidates or updates the relevant cache entry so the table reflects the new status immediately, and Ant Design shows a confirmation notification.

Cross-cutting:

- Every request/response passing through the Axios interceptors, and key cache, user-interaction, and Logic Layer events, are emitted to the observability layer described in Section 6.

5. Technology Stack & Justification

The stack was chosen to balance developer velocity for a time-boxed challenge with patterns that scale to a production system: a typed component model, a clear separation between client state and server state, a headless table for future virtualization, and a mock layer that can be swapped for a real backend with minimal code changes.

Technology	Role	Justification
React 19	UI library	Component-based architecture enabling reusable, maintainable, and scalable frontend development; industry-standard with a strong ecosystem and first-class TypeScript support.
TypeScript	Static typing	Adds compile-time type safety, strongly-typed API contracts, and earlier error detection, improving maintainability as the codebase grows.
Vite	Build tool	Instant dev-server startup and fast Hot Module Replacement (HMR) with lightweight configuration, giving a fast local feedback loop.
TanStack Table	Headless table engine	Separates table logic from presentation; supports advanced sorting/filtering and pairs with TanStack Virtual for virtualized rendering of large inventories.
TanStack React Query	Server-state management	Handles caching, background refetching, and loading/error states out of the box, reducing boilerplate and easing the move from mocks to a real API.
Redux Toolkit	Client-state management	Centralizes UI-level shared state (filters, preferences) with predictable updates and simplified store configuration.
Ant Design	Component library	Provides accessible, professional-looking components (Modal, Form, Select, Button, Notification) that speed up development of surrounding UI.
Axios	HTTP client	Centralizes the API layer with interceptors for consistent error handling and a simple path to swap mocks for production endpoints.
MSW (Mock Service Worker)	API mocking	Simulates realistic backend behavior at the network level with no dependency on a live backend, and works seamlessly with React Query.
Moment Timezone	Date/time handling	Provides reliable, UTC-based date calculations for consistent aging-stock evaluation (>90 days in inventory) across timezones.
Vitest	Test runner	Integrates natively with Vite, offering fast execution and a Jest-like API with minimal configuration.
React Testing Library	Testing utilities	Encourages behavior-driven tests from the user's perspective, reducing coupling to implementation details.

6. Observability Strategy

Even though the current backend is mocked, the application is structured so that logging, metrics, and tracing can be wired in now and connected to a real observability backend (e.g., Sentry, Datadog, Grafana/Loki/Tempo, or an OpenTelemetry collector) later with minimal rework.

Logging

- Centralize logging in the Axios interceptors: log every outgoing request (method, URL, correlation ID) and every response or error (status, duration, payload size) at the single point all HTTP traffic passes through.
- Wrap the application in a React Error Boundary that logs render-time exceptions with component-stack context, so UI failures are captured rather than silently breaking the page.
- Use structured, leveled logs (debug/info/warn/error) with a consistent JSON shape so logs can later be shipped to a log-aggregation service without reformatting.

Metrics

- Track operational metrics such as inventory fetch latency, cache hit/miss rate from React Query, count of aging-stock vehicles over time, and frequency of action-status changes.
- Capture front-end performance via Web Vitals (LCP, INP, CLS) to monitor perceived performance as the dataset and table size grow.
- Emit these as counters/gauges/histograms to a metrics endpoint (e.g., an OpenTelemetry Metrics exporter or a lightweight analytics beacon) rather than only console output.

Tracing

- Generate a correlation/request ID per user action in the Axios interceptor and attach it as a request header (W3C traceparent format), so a single user action can be followed end-to-end once a real backend participates in the same trace.
- Structure the client for future adoption of the OpenTelemetry JavaScript SDK, instrumenting React Query's fetch/mutation lifecycle as spans (start, success/error, duration).
- Keep trace/log correlation IDs visible in development (e.g., console grouping) to speed up local debugging before a full tracing backend exists.

7. Trade-offs & Future Scalability Considerations

This section was not fully specified in the original prompt; it is included as a reasonable interpretation of a typical closing section for a system design document, covering the trade-offs made for this challenge and the path to a production-scale system.

- Client-side filtering and pagination were chosen for simplicity; a production system with thousands of vehicles would move to server-side pagination, filtering, and sorting, which TanStack Table and React Query already support without a rewrite.
- Redux Toolkit and React Query intentionally do not overlap: Redux owns UI-only state (filters, preferences) while React Query owns anything that originates from the server, avoiding duplicated or stale state.
- MSW's handler-based design means the mock-to-real-backend migration is a configuration change at the network boundary, not a rewrite of components or hooks.
- TanStack Table's headless architecture, combined with TanStack Virtual, gives a clear upgrade path to virtualized rendering once inventory volume grows beyond what unvirtualized rendering can handle smoothly.
- Planned but out of scope for this challenge: role-based access control, an audit history for action-status changes, a dedicated vehicle detail page, Docker and CI/CD integration with automated end-to-end tests (Playwright).